AED2 - Atividade Teórica - Aula 4

Aluno: João Marcelo Franca Batista

1) Ordenação por contagem (Counting Sort) — A = [6,0,2,0,1,3,4,6,1,3,2]

Passo 1 — Contar quantas vezes cada valor aparece em A

Valor	0	1	2	3	4	5	6
Contagem C	2	2	2	2	1	0	2

Passo 2 — Somar as contagens de forma acumulada (soma de prefixos)

Valor	0	1	2	3	4	5	6
Acumulado C'	2	4	6	8	9	9	11

C'[valor] indica em qual posição (contando a partir de 1) termina o bloco daquele valor no vetor final.

Passo 3 — Colocar cada elemento de A na posição C'[valor] (da direita para a esquerda, para manter a ordem original entre iguais)

i	A[i]	Posição usada (C'-1)	C' após decrementar
10	2	5	5
9	3	7	7
8	1	3	3
7	6	10	10
6	4	8	8
5	3	6	6
4	1	2	2
3	0	1	1
2	2	4	4
1	0	0	0
0	6	9	9

Resultado final

```
A ordenado = [ 0, 0, 1, 1, 2, 2, 3, 3, 4, 6, 6 ]
```

2) Radix Sort de palavras (3 letras cada)

Lista original: cow, dog, sea, rug, row, mob, box, tab, bar, ear, tar, dig, big, tea, now, fox.
O Radix Sort para strings de tamanho fixo ordena, de forma ESTÁVEL, letra por letra, começando pela última (menos significativa) até a primeira (mais significativa).

Passada 1 — ordenar pela 3ª letra (última)

```
sea, tea, mob, tab, dog, rug, dig, big, bar, ear, tar, cow, row, now, box, fox
```

Passada 2 — ordenar pela 2ª letra (letra do meio)

`tab, bar, ear, tar, sea, tea, dig, big, mob, dog, cow, row, now, box, fox, rug`

Passada 3 — ordenar pela 1ª letra

`bar, big, box, cow, dig, dog, ear, fox, mob, now, row, rug, sea, tab, tar, tea`

Resultado final (ordem alfabética)

`bar, big, box, cow, dig, dog, ear, fox, mob, now, row, rug, sea, tab, tar, tea`

3) Algoritmo mais eficiente para cada caso

Considerando os algoritmos vistos até agora (Bubble Sort, Selection Sort, Insertion Sort, Counting Sort e Radix Sort):

a) Lista já ordenada em ordem ascendente

Insertion Sort. É o melhor caso dele: como cada elemento já está na posição certa em relação aos anteriores, ele só faz 1 comparação por elemento e nenhuma troca — complexidade $\Omega(n)$. Além disso usa memória extra $O(1)$ (ordena no próprio vetor), menos memória do que Counting/Radix Sort, que precisam de vetores auxiliares.

b) Lista já ordenada em ordem descendente

Esse é o pior caso para os algoritmos de comparação: Insertion Sort e Bubble Sort chegam ao seu $O(n^2)$ máximo (o Insertion Sort precisa mover cada elemento até o início). O Selection Sort não piora nem melhora, pois sempre faz $O(n^2)$ comparações independente da ordem de entrada — por isso, entre os algoritmos de comparação, é o mais previsível aqui. Mas se os valores tiverem uma faixa conhecida e limitada (como as datas da questão prática), Counting Sort ou Radix Sort são melhores: $O(n)$, pois não dependem da ordem de entrada.

c) Lista com valores desordenados aleatoriamente

Esse é o caso médio $\Theta(n^2)$ para Bubble, Insertion e Selection Sort — nenhum tem vantagem clara de tempo entre si. Ainda assim, o Selection Sort tende a fazer menos trocas (no máximo n), o que ajuda se cada troca for cara. De novo, se o intervalo de valores for pequeno e conhecido, Counting Sort/Radix Sort ($O(n)$) são mais eficientes que qualquer um dos algoritmos de comparação.

d) Elementos com valores iguais não podem trocar de ordem (estável)

É preciso usar um algoritmo estável: Insertion Sort, Bubble Sort, Counting Sort ou Radix Sort (quando construído com Counting Sort). O Selection Sort NÃO é estável, pois troca elementos que podem estar distantes um do outro no vetor, o que pode inverter a ordem de elementos iguais. Entre os estáveis, Counting/Radix Sort ($O(n)$) são os mais eficientes quando o intervalo de valores é pequeno; caso contrário, Insertion Sort é uma boa opção estável e simples.